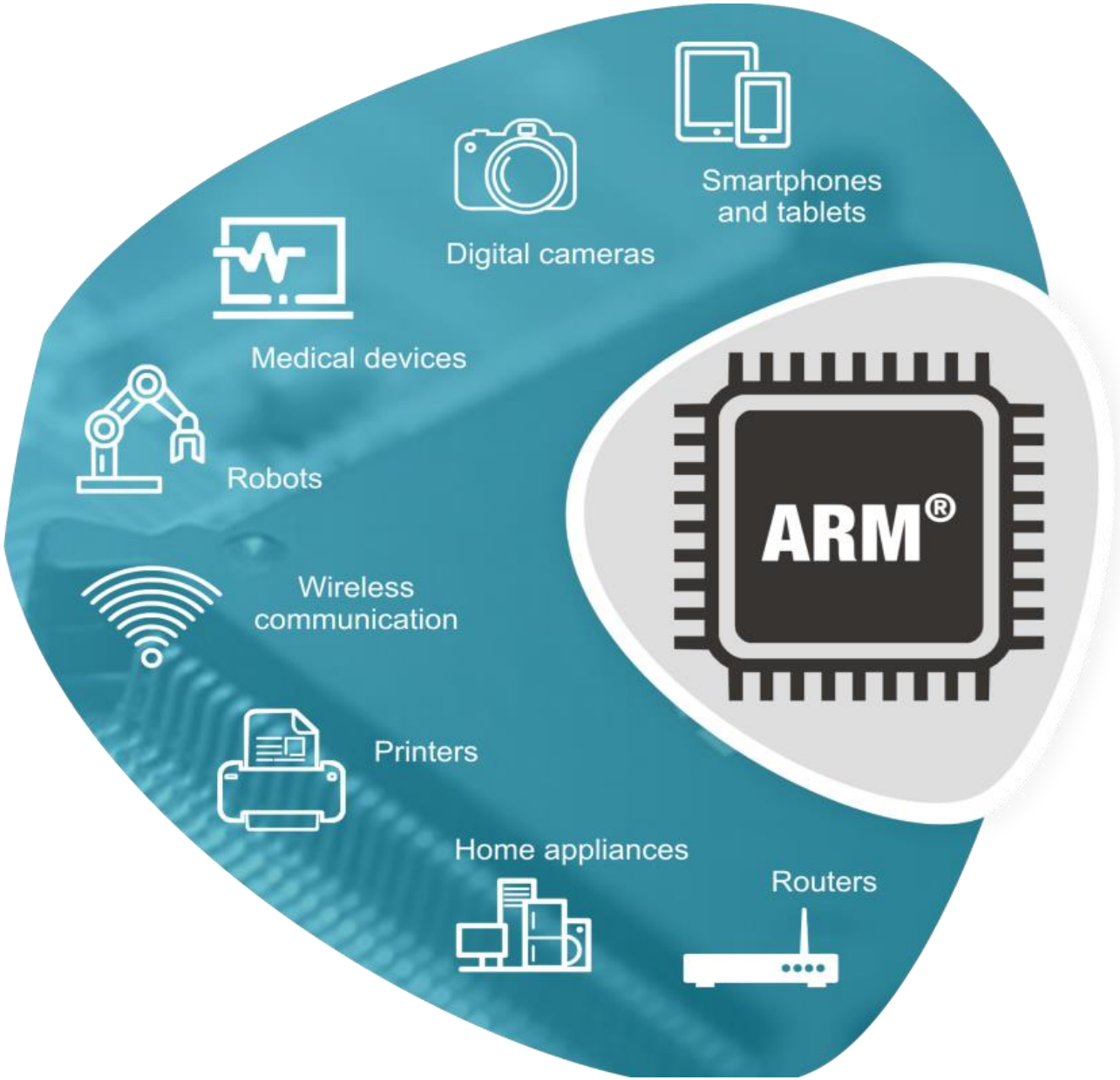
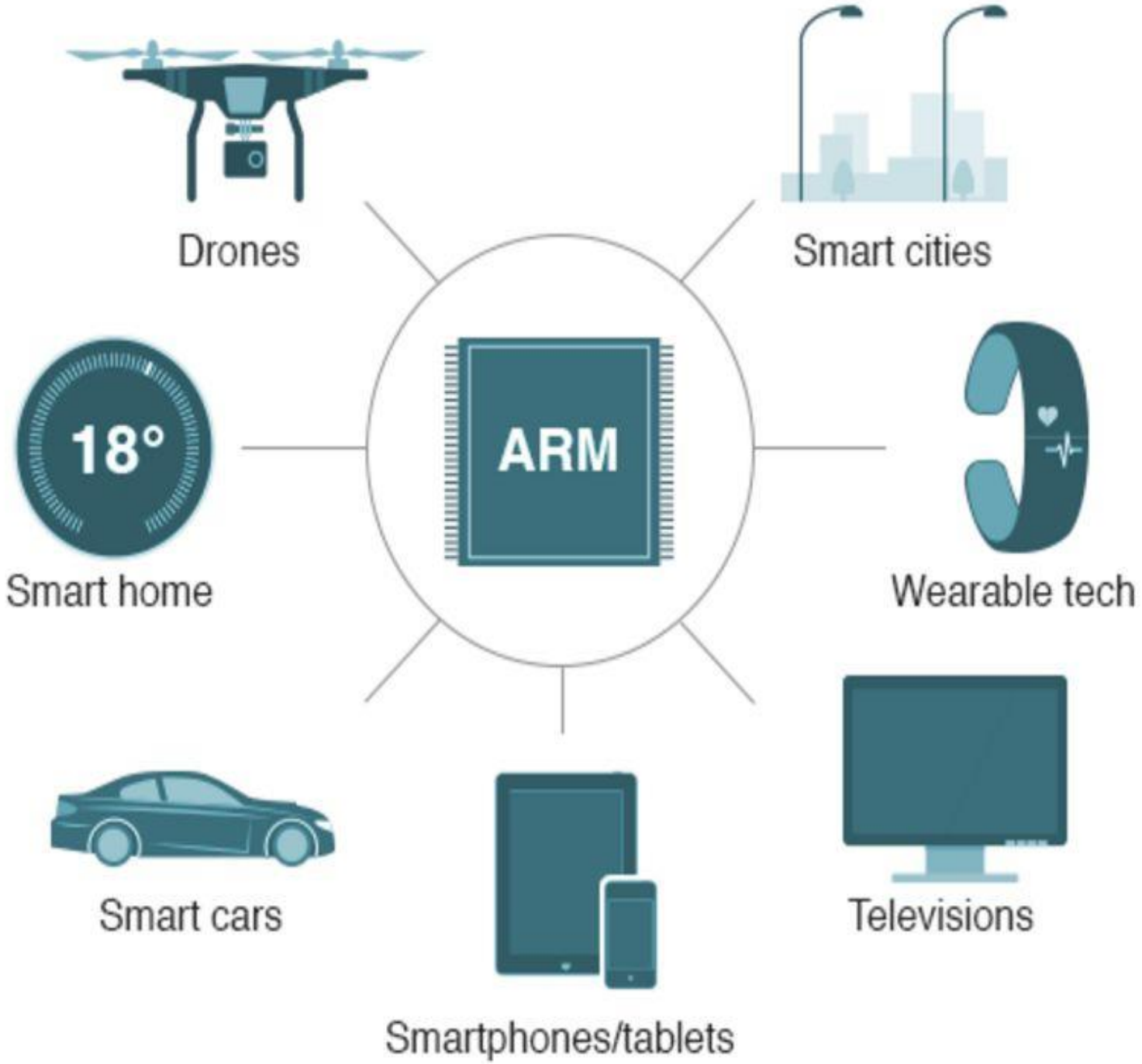


Embedded Systems

- ARM - Stands for Advanced RISC Machines.
- ARM - accounts for more than 80% of embedded products in the economy.

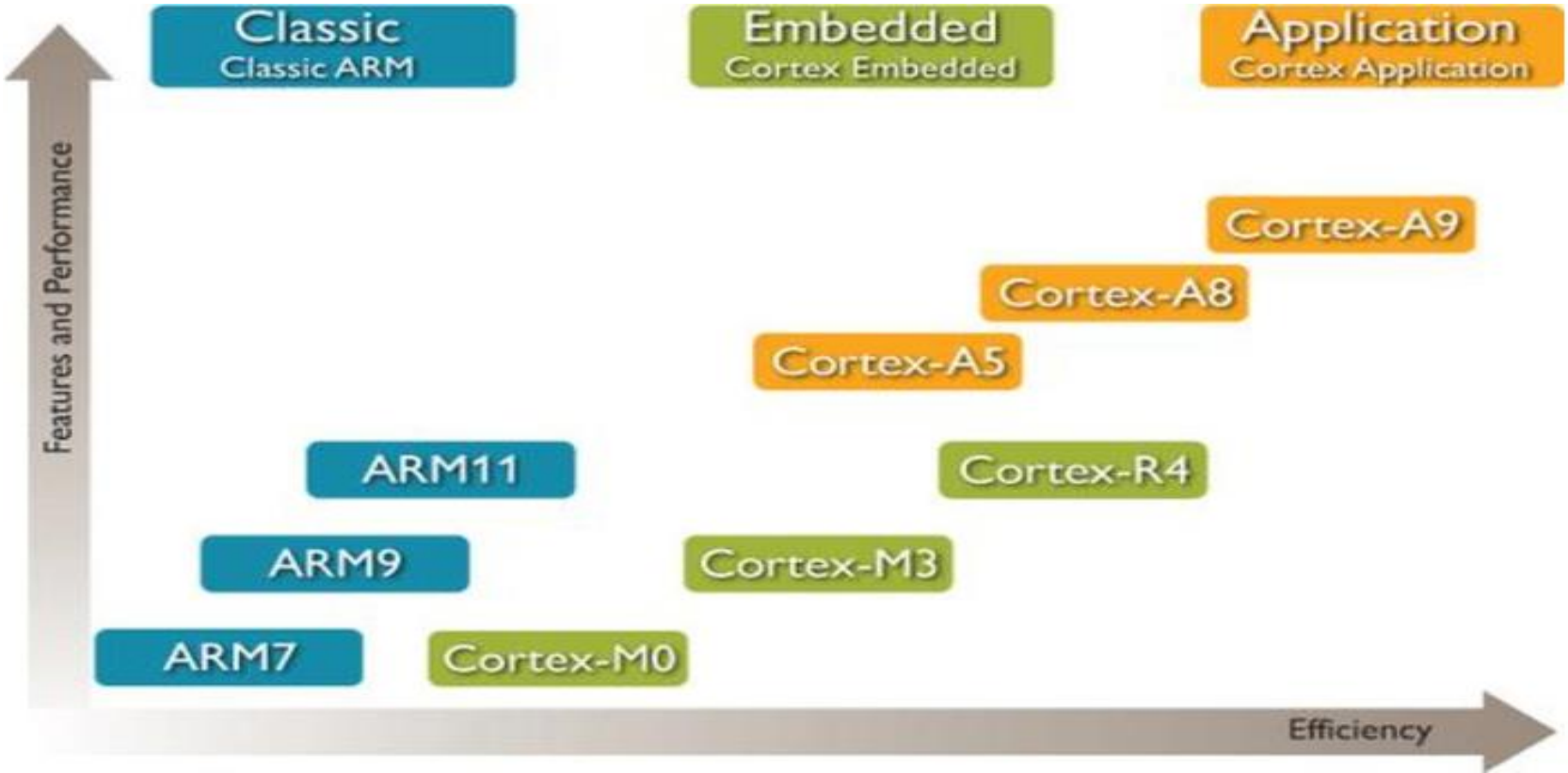
Applications of ARM




Drones
Smart cities
18°
ARM
Smart home
Wearable tech
Smart cars
Smartphones/tablets
Televisions

ARM History

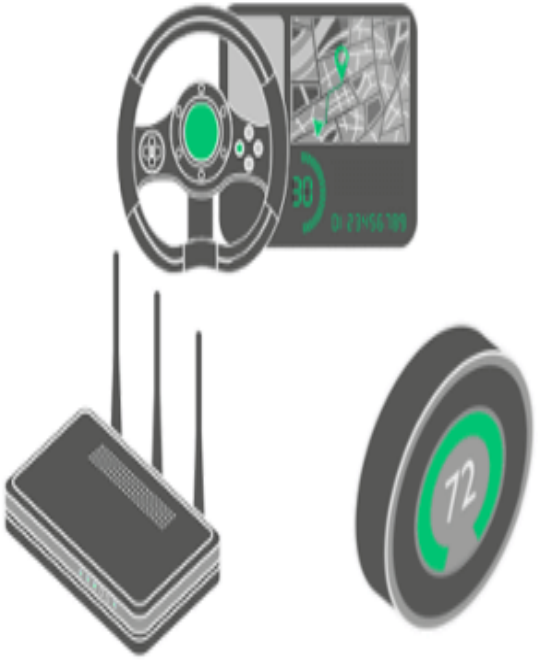
- Developed by **Acorn Computers Limited**, Cambridge early 80's
  - **First commercial RISC Processor**
- Renamed **Advanced RISC Machines (ARM) Limited** in the early 90's
- ARM Core is not a single core
- Evolution from ARM1 to ARM Cortex (A,R,M) Series Over the Years.



Cortex - A

Highest performance

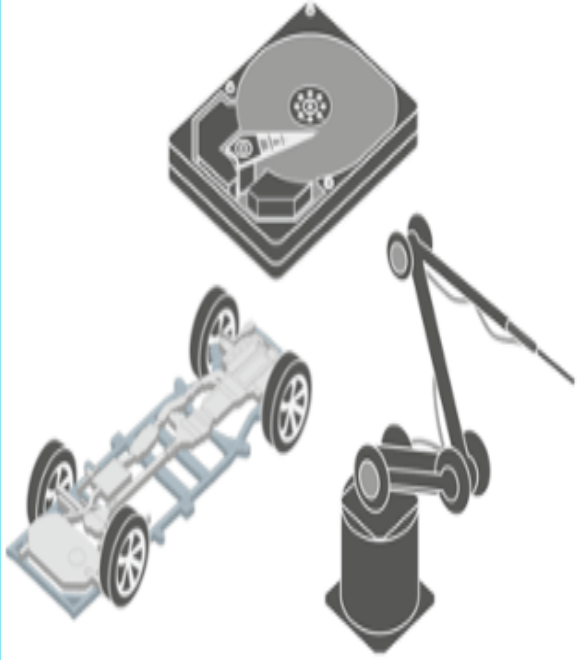
Optimised for
rich operating systems



Cortex - R

Fast response

Optimised for
high performance,
hard real-time applications



Cortex - M

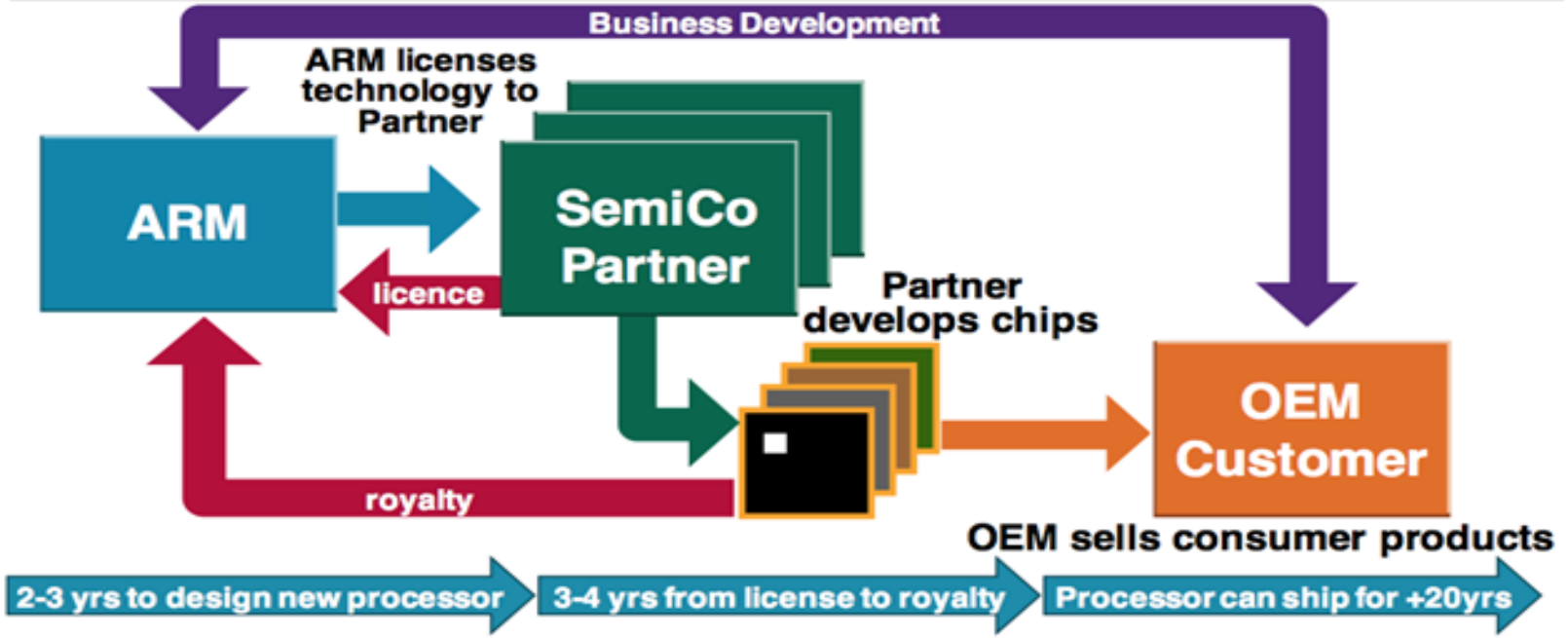
Smallest/lowest power

Optimised for
discrete processing and
microcontrollers



- ARM has been licensed to many semiconductor manufacturers like:
  - **National Semiconductor**
  - **Philips**
  - **Atmel**
  - **Texas Instrument (TI)**
  - **ST Microelectronics**
  - **Qualcom**
  - **etc**

ARM Business Model

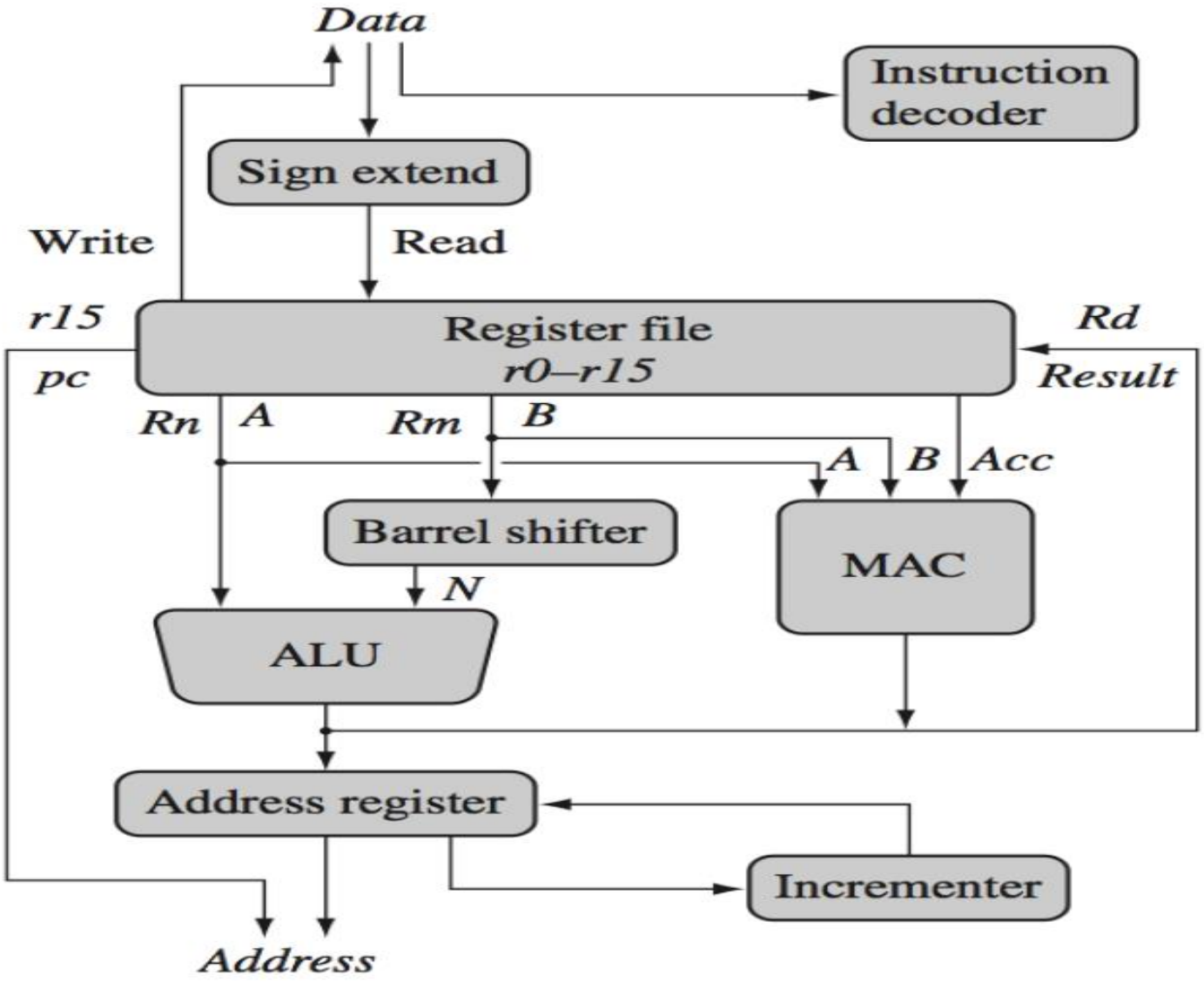


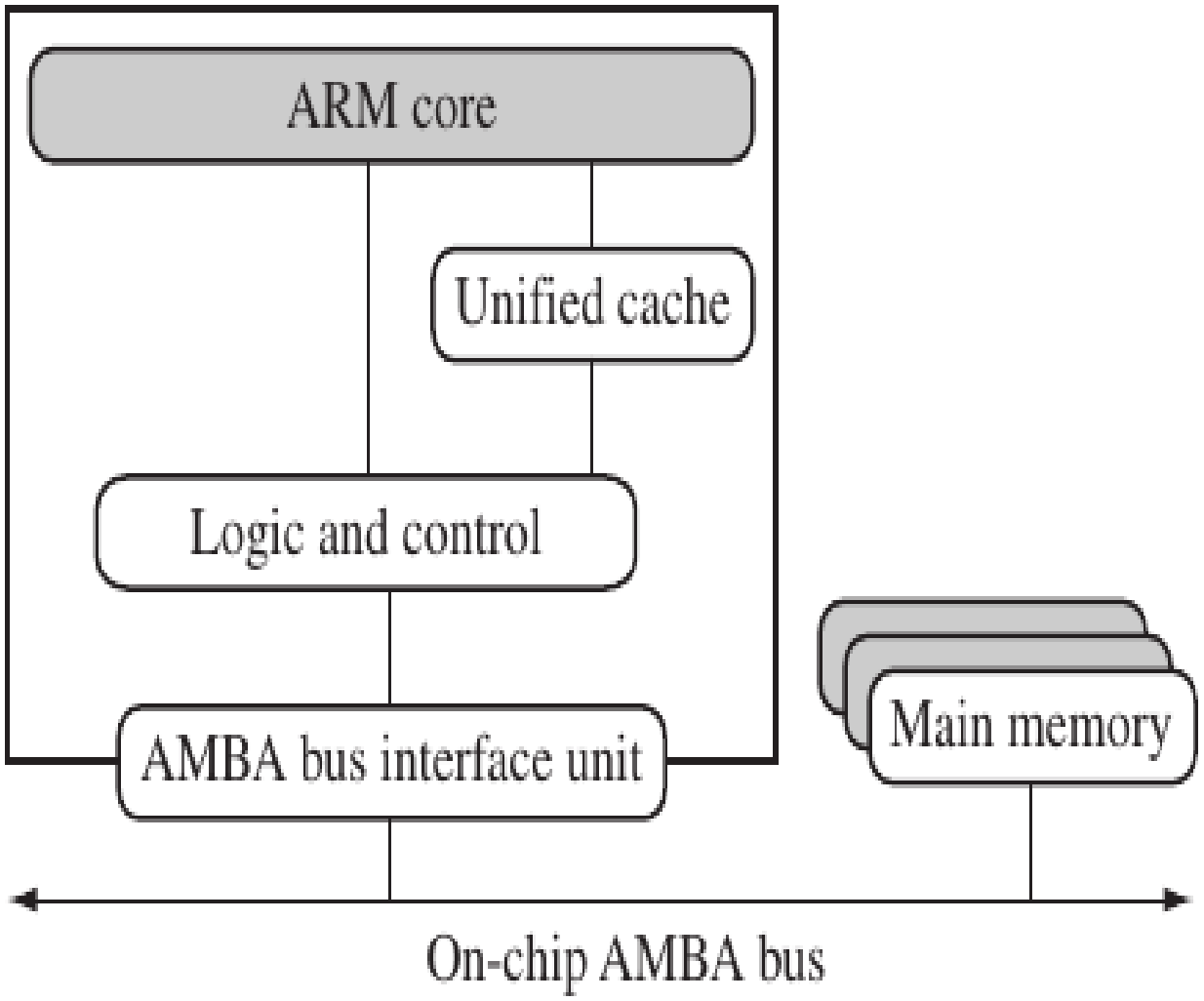
- Innovative business model yields high margins
  - Upfront license fee – flexible licensing models
  - Ongoing royalties – typically based on percentage of chip price
  - Technology suitable for multiple applications – can ship for decades

ARM Core (Data Flow Model)

- **An ARM core can be viewed as functional units connected by buses**
- **The information on bus may be an instruction or a data item or an address**

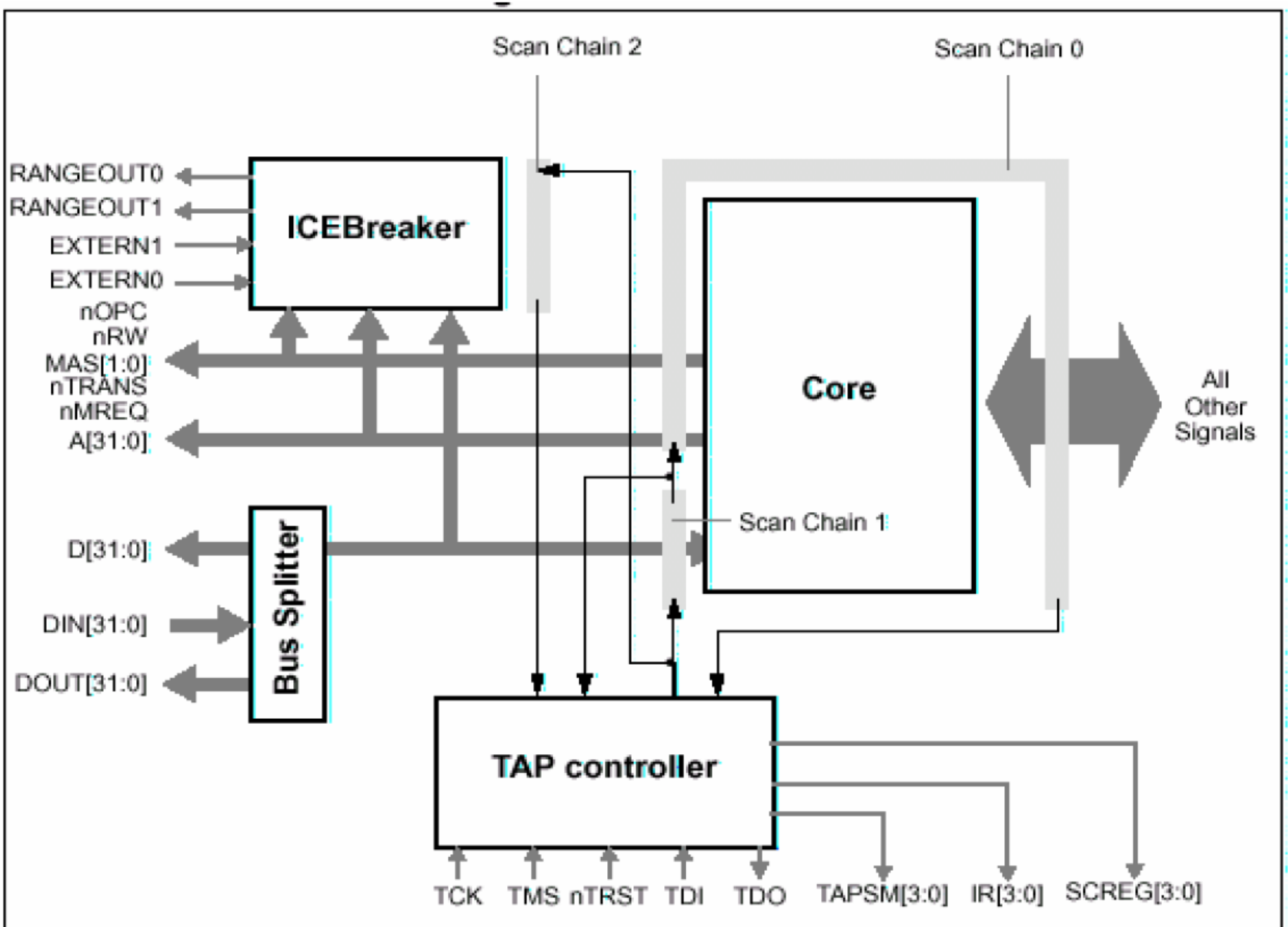
The Core (Data Flow Model)

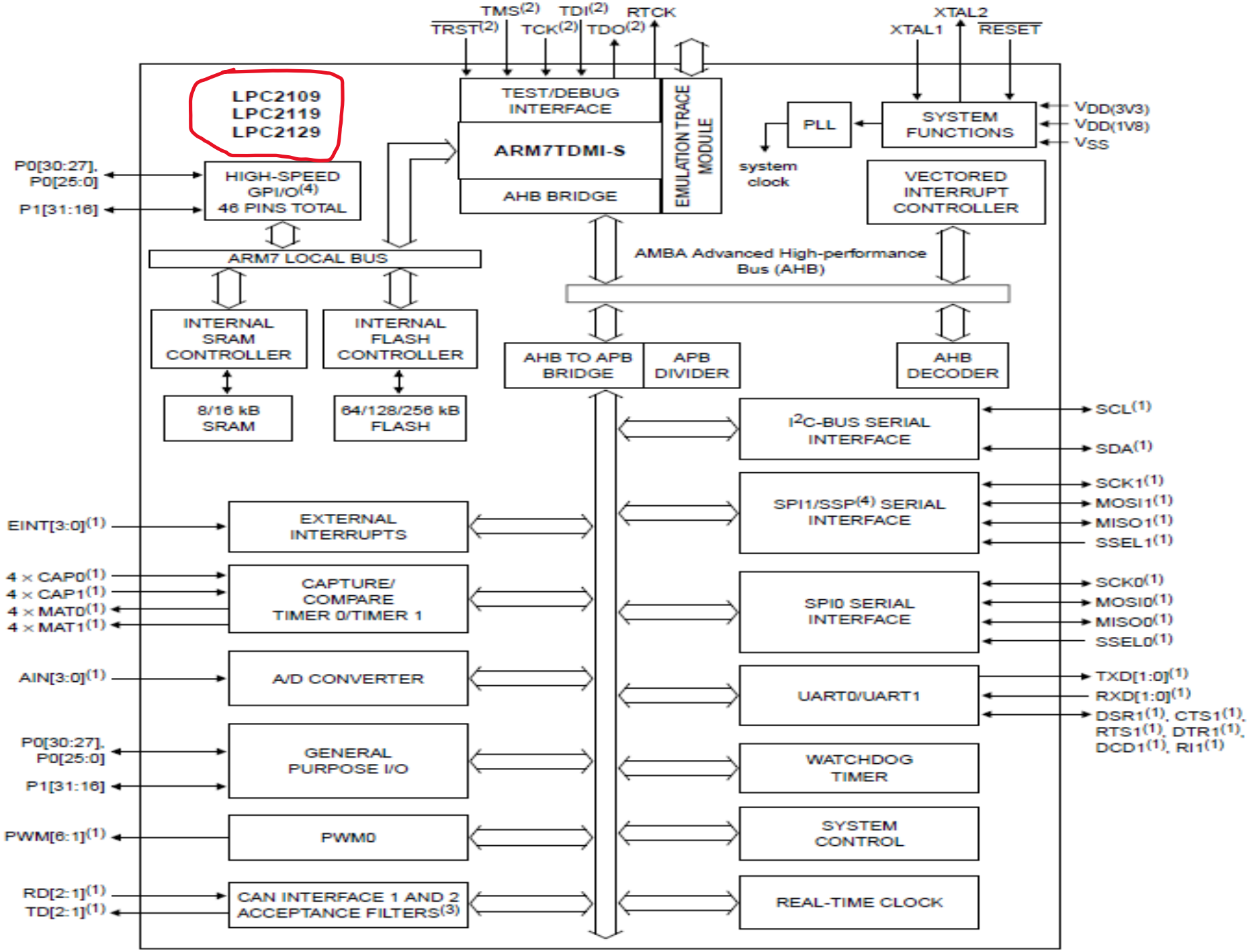




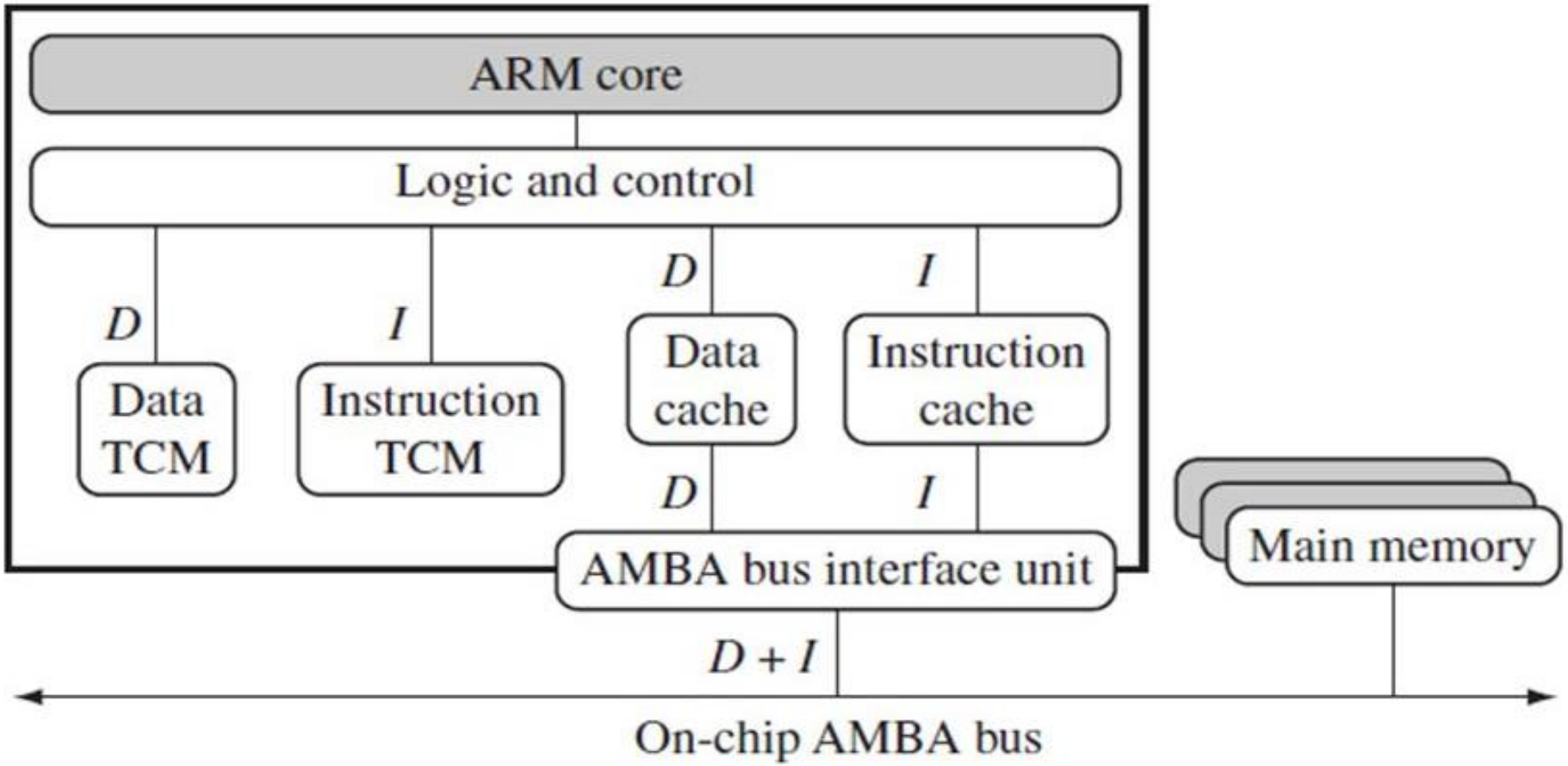
A simplified Von Neumann architecture with cache.

ARM7TDMI Block Diagram




LPC2109
LPC2119
LPC2129
TRST(2)
TMS(2)
TCK(2)
TDI(2)
TDO(2)
RTCK
TEST/DEBUG INTERFACE
ARM7TDMI-S
AHB BRIDGE
EMULATION TRACE MODULE
XTAL1
XTAL2
RESET
PLL
SYSTEM FUNCTIONS
system clock
VDD(3V3)
VDD(1V8)
VSS
VECTORED INTERRUPT CONTROLLER
P0[30:27], P0[25:0]
P1[31:16]
HIGH-SPEED GPI/O(4) 46 PINS TOTAL
ARM7 LOCAL BUS
AMBA Advanced High-performance Bus (AHB)
INTERNAL SRAM CONTROLLER
INTERNAL FLASH CONTROLLER
8/16 kB SRAM
64/128/256 kB FLASH
AHB TO APB BRIDGE
APB DIVIDER
AHB DECODER
I2C-BUS SERIAL INTERFACE
SCL(1)
SDA(1)
EINT[3:0](1)
EXTERNAL INTERRUPTS
SPI1/SSP(4) SERIAL INTERFACE
SCK1(1)
MOSI1(1)
MISO1(1)
SSEL1(1)
4 × CAP0(1)
4 × CAP1(1)
4 × MAT0(1)
4 × MAT1(1)
CAPTURE/ COMPARE TIMER 0/TIMER 1
SPI0 SERIAL INTERFACE
SCK0(1)
MOSI0(1)
MISO0(1)
SSEL0(1)
AIN[3:0](1)
A/D CONVERTER
UART0/UART1
TXD[1:0](1)
RXD[1:0](1)
DSR1(1), CTS1(1), RTS1(1), DTR1(1), DCD1(1), RI1(1)
P0[30:27], P0[25:0]
P1[31:16]
GENERAL PURPOSE I/O
WATCHDOG TIMER
PWM[6:1](1)
PWM0
SYSTEM CONTROL
RD[2:1](1)
TD[2:1](1)
CAN INTERFACE 1 AND 2 ACCEPTANCE FILTERS(3)
REAL-TIME CLOCK

ARM Core (Harvard Model)



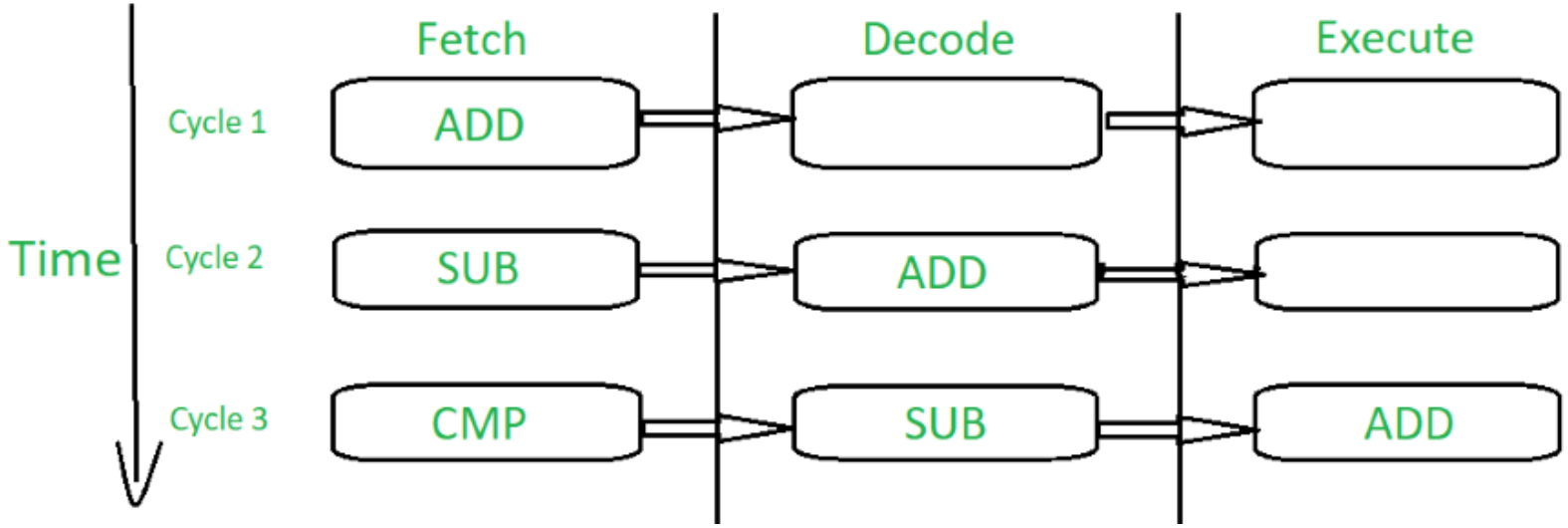
A simplified Harvard architecture with caches and TCMs.

ARM 7 Architectural Features

- **Core uses a Von Neuman & RISC ISA**
- **32-bit core**
- **32-bit ALU**
- **32-bit instructions,**
- **32-bit Address Bus,**
- **32-bit Data Bus With Aligned Memory Space,**
- **37, 32-bit core registers (16 available/mode –register file)**
- **Features High Code density (2 ISAs : ARM(32) & THUMB(16))**
- **8-bit(byte) / 16-bit(halfword) / 32-bit(word) data types**
- **Words are always aligned at 4-byte boundaries**
- **Half Words are aligned on even byte boundaries**
- **Like any other RISC arch,uses Load-Store Instruction Set.**

  (LOAD : REG ← MEM,STORE :REG → MEM)

- **Uses 3 Stage Pipeling (Fetch,Decode & Execute Overlapped)**



- **Coincidently ! Core Supports 7 Modes of Core Operations**
- **Coincidently ! Core Support 7 Addressing Modes**
- **Coincidently ! Core Support 7 Sources of Interrupts/Execeptions**

ARM 7 : 7 Processor Modes

- **The processor mode determines which registers are active and the access rights to the cpsr(flag) register itself.**
- **A privileged mode allows full read-write access to the cpsr**
- **A nonprivileged mode only allows read access to the control field in the cpsr, but still allows full read-write access to the condition flags**
- **Six privileged modes**

  Abort (abt),

  fast interrupt request (fiq),

  interrupt request (irq),

  supervisor (svc),

  system (sys),

  undefined (und)
- **One nonprivileged mode**

  user (usr)

1. User Mode :

- This is the normal mode in which all user programs are executed
- It is the only non-priviledged mode
- Has limited access to flags in CPSR flag register

2. Fast Interrupt Request Mode :

- This mode is entered when a high priority interrupt occurs through nFIQ pin
- In this mode the ISR executes with minimum latency(delay) due less context switching(saving/pushing & retrieving/popping) time.
- Nesting of interrupts are disabled in FIQ mode

3. Interrupt Request Mode :

- This is normal Interrupt Mode and is invoked when a low priority interrupt occurs on the nIRQ pin.
- Here interrupt latency is more and nested interrupt are allow

4. Supervisor Mode :

- ARM 7 enters this mode on reset/power up.
- It is used to execute the BIOS/Bootloader Program.
- Is generally the mode that an operating system kernel operates in.
- This mode can also be invoked by the programmer by SWI (Software Interrupt) Instruction.

5. Abort Mode :

- This mode is entered when an unsuccessful attempt is made to access a memory location

6. Undefined Mode :

- When the processor encounters an instruction that is undefined or not supported by the implementation
- Co-processors connected to ARM7 processor will operate in this mode

7. System Mode :

- This is privileged version of the User Mode.
- The User Program can invoke this mode to gain full control (read-write) over CPSR.

ARM is not pure RISC.
Its instruction SET Differs from pure RISC as:

- Variable cycle execution for certain instructions
- Inline barrel shifter leading to more complex instructions
- Thumb 16 bit instructions set
- Conditional execution
- Enhanced (DSP) instructions
- Jazelle Support (70 % Java Byte Codes)

Inline barrel shifter impact

8051 Arch .Operation

z=((x<<4)+ y)	S	T
1) MOV A,R7	1	1
2) CLR C	1	1
3) RLC A	1	1
4) CLR C	1	1
5) RLC A	1	1
6) CLR C	1	1
7) RLC A	1	1
8) CLR C	1	1
9) RLC A	1	1
10)ADD A,R6	1	1
11)MOV R5,A	1	1

OR ALSO	S	T
1)MOV A,R7	1	1
2)MOV B,#16	3	2
3)MUL AB	1	4
4)ADD A,R6	1	1
5)MOV R5,A	1	1

While in ARM 7 it is accomplished with ONLY 1 instruction!

- ADD R5,R6,R7,LSL #4